

Tablets and Vnodes During Cluster Expansion and Contraction in ScyllaDB

An instrumented single-host comparative case study

Giorgi Gogsadze

Department of Computer Science, Kutaisi International University, Georgia

24 September 2026

Abstract

Cluster expansion and contraction couple replica placement, storage maintenance and foreground service. We examine these interactions through one tablet lifecycle and one vnode lifecycle in ScyllaDB on a resource-constrained laptop. Each RF=2 dataset contained 750,000 deterministic rows and traversed a three-to-four-to-three-node sequence. The treatments executed identical sequences of 1.08 million measured operations at a target of 500 operations/s, with zero reported workload failures. Tablets split from 32 to 64 ranges and merged back, achieving equal four-node token-space shares while changing some final replica assignments. Client backlog was largest during tablet expansion and several minutes after vnode contraction; the latter coincided with synchronized deferred storage work. Topology quiescence preceded this delayed slowdown, illustrating the need to evaluate placement and service separately. An offline SSTable audit accounted for all 72,501 excess final vnode entries as overlap across local files: exactly 750,000 distinct keys remained, each on two nodes. Together, these findings connect placement transitions, maintenance and physical accounting. Unequal baselines, asymmetric log retention and one lifecycle per treatment constrain causal and statistical generalization.

Keywords: ScyllaDB; tablets; virtual nodes; topology changes; streaming; compaction; measurement validity

1 Introduction

Adding or removing a database node is not a single event. Membership changes, replicas move, obsolete files are reclaimed, and clients continue to issue operations while the storage engine reorganizes data. A cluster can therefore appear stable in its topology metadata while foreground performance or local disk activity remains unsettled. Comparisons that report only operation latency or the return time of a topology command can miss those distinctions.

ScyllaDB offers two data-distribution mechanisms within the same database: vnode-based token ownership and table-specific tablets with explicit replica placement [1, 2]. This makes it possible to compare mechanisms while retaining the database implementation, schema, replication factor and workload. However, their monitoring surfaces are not interchangeable. A tablet replica-size observation, a vnode ownership percentage, a stream-manager byte counter and an SSTable partition estimate describe different quantities.

This paper addresses three questions. First, how do placement and observed quiescence evolve through a complete expansion/contraction lifecycle? Second, how do client scheduling, operation latency and storage maintenance relate to those transitions? Third, which apparent differences survive an audit of metric meaning, sampling and physical storage contents?

The contribution is an event-aligned case study combining request logs, continuous topology and resource observations, retained vnode server logs, exact-build source inspection and an offline audit of preserved SSTables. It reconstructs tablet placement inheritance, separates topology quiescence from delayed service effects, and accounts exactly for an apparent excess of physical replicas. These complementary outcomes provide a baseline for studying scaling mechanisms under constrained resources.

2 Background and related work

Dynamo’s consistent-hashing design assigns multiple virtual positions to physical nodes to support incremental redistribution [3]. Apache Cassandra’s architecture documentation describes the balance and operational tradeoffs of multiple tokens, including the larger set of replica neighbors [4]. Equal token counts therefore need to be distinguished from effective replicated ownership. Bigtable develops tablets as independently managed row ranges [5]; ScyllaDB applies table-specific partitioning to a replicated token space, with explicit host and shard placement [1].

ScyllaDB’s implementation account describes tablets as units of placement and storage organization, including per-tablet memtables and SSTables, migration and cleanup [6]. Its Raft-topology account explains the coordination of metadata changes through a fault-tolerant topology coordinator [7]. These sources motivate inspecting both range boundaries and replica assignments. They also clarify the comparison: both treatments here run the same ScyllaDB build, so the contrast concerns data distribution within ScyllaDB. Cassandra studies provide context rather than an interchangeable implementation baseline.

Elasticity research evaluates the cost of changing capacity while serving requests. Kuhlenkamp, Klems and Röss compare Cassandra and HBase using both scaling speed and workload impact, and examine the sensitivity of results to infrastructure and reproduction [8]. Yamaguchi and Morimitsu identify disk I/O during Cassandra node joining as a scaling bottleneck and investigate page-cache and I/O-scheduling changes [9]. Their findings motivate examining storage work alongside topology, without identifying the cause of an individual slowdown in our environment. OnlineElastMan uses workload prediction and a learned performance model to provision Cassandra storage under changing load and response-time objectives [10]. Its proactive control problem complements our fixed, externally triggered lifecycle, which concentrates on explaining immediate and delayed effects.

YCSB supplies a configurable framework for cloud-serving evaluation [11]. Our custom serial client instead preserves identical operation/key sequences and records scheduled and actual starts, making accumulated backlog directly auditable. Offline storage inspection adds another perspective: ScyllaDB’s partition-estimate path aggregates SSTable estimates [12], while repeated keys across files can inflate the total relative to distinct replicas. Flushing persists memtables while leaving other aspects of memory state intact [13]. Accordingly, we analyze logical ownership, physical file overlap, maintenance and client service as separate outcomes.

3 Experimental design

3.1 Testbed and controls

Table 1: Experimental configuration and available hardware evidence.

Component	Configuration
Host	Windows laptop, 16 GB RAM; Docker-managed Linux VM
Processor	Intel Core i5-13450HX, x86-64; 1 socket, 10 cores (6 performance + 4 efficient), 16 logical processors
Nominal clocks	Author-reported base 2.40 GHz; manufacturer maximum turbo 4.60 GHz [14]
Storage	NVMe SSD, SOLIDIGM SSDPFINW010TZL
VM allocation evidence	Approximately 9.711 GiB displayed as the common Docker memory ceiling in both treatments; configured memory and vCPU limits unavailable
Database	ScyllaDB 2026.3.0-0.20260828.10c95df28f74
Per-node settings	Two shards, <code>--memory 2G</code> , developer mode, overprovisioned
Topology	One datacenter, one rack; initially three nodes
Replication	NetworkTopologyStrategy, RF=2, durable writes
Treatments	Tablets enabled; tablets disabled with 256 vnodes per node
Schema	<code>id bigint PRIMARY KEY, value text</code>
Storage settings	IncrementalCompactionStrategy; LZ4WithDictsCompressor; caching ALL
Dataset	750,000 IDs, 0-749,999; deterministic 1,024-character values; seed 42
Workload	One synchronous client; target 500 operations/s; 80% reads
Consistency	LOCAL_QUORUM for reads and writes

Hardware identity and base speed were supplied retrospectively by the author; virtualization was enabled.

The maximum turbo specification is a hardware capability, not a measured experiment clock [14]. The recorded Docker memory ceiling is rounded and does not establish an exact VM configuration. Two Scylla shards and `--memory 2G` describe database process settings, not the VM’s vCPU count or a container hard-memory quota. Effective CPU frequency, storage throughput and VM vCPU allocation were not recorded in the available primary evidence.

The exact version and build ID match ScyllaDB’s published x86-64 **release** entry [15]. The date/commit suffix identifies its build; it does not identify a pre-release. Runtime developer mode is a separate setting. The schema and `LOCAL_QUORUM` settings follow ScyllaDB’s documented interfaces [16, 17].

The treatments used fresh project-scoped volumes and distinct keyspaces. Tablets ran first on 21 September 2026 and vnodes second on 22 September. Fixed container names and network addresses required sequential execution. The dataset remained loaded through each lifecycle, but workload phases were separated by checks, warmups and operator delays. Thus the experiment maintained dataset continuity, not uninterrupted mixed traffic across the entire day.

RF=2 preserves nontrivial replica-placement choices in a three-node baseline. The one-rack tablet configuration does not satisfy ScyllaDB’s RF-rack-valid terminology [18], and a separate creation warning reported RF below the recommended threshold of three. The configuration permits the placement and streaming observations studied here; the findings apply to this single-rack warning state. Rack-failure behavior was outside the experiment.

The observed startup warnings also matter: the perf-event stall detector fell back to a timer, `SCHED_FIFO` scheduling was unavailable, memory was approximately 974 MiB per shard, and the I/O scheduler was reported as improperly configured. These deviations from recommended deployment conditions affect the interpretation of performance [19].

3.2 Lifecycle and workload matching

Table 2: Measured lifecycle phases; each phase is preceded by a read-only warmup.

Phase	Nodes	Duration (s)	Operations	Seed	Topology action
Baseline	3	120	60,000	20260921	None
Expansion	3 to 4	900	450,000	20260922	Add node at scheduled +120 s
Four-node	4	120	60,000	20260923	None
Contraction	4 to 3	900	450,000	20260924	Decommission at scheduled +120 s
Final three	3	120	60,000	20260925	None

Every phase had a 30-second, read-only warmup at the same target rate. Measured workloads selected IDs uniformly and generated deterministic updates. All 1,080,000 corresponding operation-type/ID pairs matched between treatments, and all eight recorded source hashes agreed. Successful writes were checked against saved latest expected seeds. The final tracked set contained 186,933 distinct written IDs in each treatment.

There were two relevant departures from a perfectly matched schedule. The vnode expansion warmup was repeated after an initially slower run; both warmups were excluded from measured results. In addition, serial cleanup was run on the original vnode nodes after measured expansion and before the four-node phase. No equivalent manual cleanup was run for tablets. Cleanup changed the storage state inherited by subsequent phases.

The study comprises one independently initialized lifecycle per treatment. Requests are observations within each lifecycle, rather than independent treatment replications; the analysis reports descriptive statistics without treatment-level significance tests.

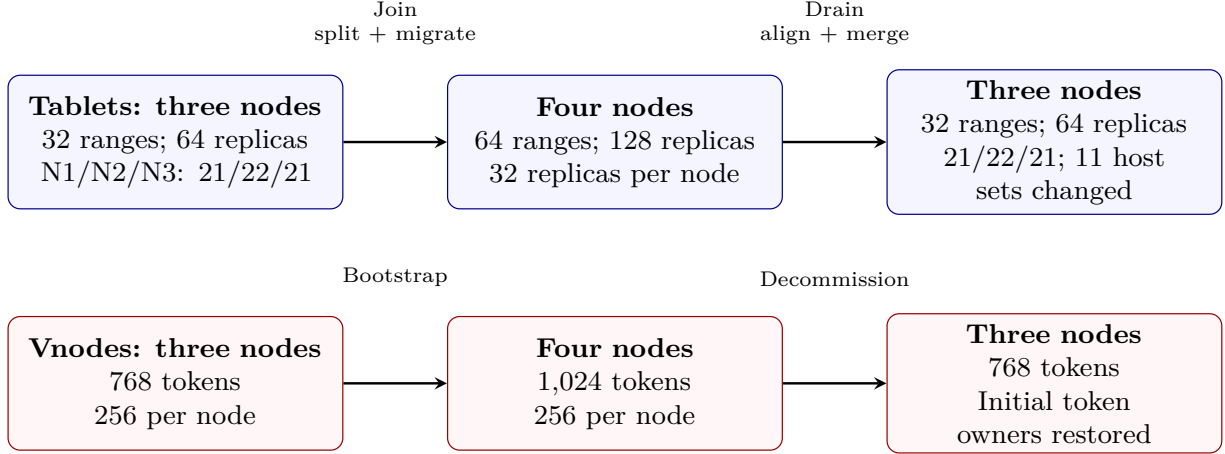


Figure 1: Observed membership and range organization. Replica counts count host/shard assignments at RF=2; arrows summarize transitions that included migration. The return to 32 tablet boundaries preserved aggregate counts but changed 11 host sets. Vnode token-owner signatures returned to their initial state.

3.3 Measurement definitions

Client operation latency is the monotonic elapsed time inside the workload’s operation boundary. That boundary includes client-side value generation, the driver call, read validation or expected-write-state handling. It is not isolated server execution time.

Scheduled-start lag is actual monotonic start offset minus scheduled offset. The client is serial: when operation time and intervening work exceed the two-millisecond average budget, it accumulates backlog and subsequently catches up. A target of 500/s therefore does not mean an independent concurrent arrival process delivered exactly 500 requests in every wall-clock second.

Placement-and-counter quiescence onset is the earliest saved topology observation followed by at least 60 seconds of unchanged placement and idle collected streaming counters, with complete required groups. It is a retrospective metadata-and-counter milestone. It does not measure all transfer paths, deferred compaction or sustained client recovery. The full confirmation window becomes available only later; collection cadence limits temporal resolution.

Ownership denotes logical token-space coverage. **Replica-size bytes** describe reported tablet replica storage. **SSTable entry counts** and **distinct local keys** are separate measures. Gross Docker network and block I/O include other traffic and storage work and are rounded cumulative values [20]. Throughout, MB denotes decimal megabytes; MiB and GiB denote binary units.

3.4 Instrumentation and analysis

Tablet placement and size metadata were sampled approximately once per second. Vnode ring commands ran approximately every two seconds, retaining raw output, acquisition start/end times and SHA-256 hashes. The documented incoming/outgoing stream-manager counters [21] were collected per node, shard and direction at roughly one-second intervals. Docker samples were typically three to four seconds apart. Event records identified command starts, returns and process removal; separate health checks recorded membership, tasks, streams, restarts and OOM flags.

Phase summaries use linear-interpolated quantiles at position $(n-1)p$, rounded to three decimals. UTC timestamps align independently recorded evidence; durations and scheduling conclusions use monotonic fields. Appendix A contains clock, sampling and parser validation details.

To investigate the final partition estimates reported by `tablestats` [22], we mounted preserved vnode volumes read-only and used the matching `dump-statistics` and `dump-data` tools [23]. File inventory and modification times selected the 12 SSTables corresponding to the pre-shutdown checkpoint. We compared metadata row counts with extracted keys, deduplicated within each node and counted node membership for every key. This retrospective audit tests whether apparent excess replicas arise from local file overlap; its selected-file scope excludes later shutdown files and memtables.

4 Results

4.1 Integrity and foreground operation times

Each treatment completed 1.08 million measured operations with zero reported failures, invalid IDs or index errors. The read/write counts matched exactly by phase: 48,170/11,830; 360,305/89,695; 48,012/11,988; 360,268/89,732; and 48,084/11,916. Saved write-state checks found no missing or mismatched latest seeds. Both final logical counts were 750,000; 500 written and 500 never-written sampled values per treatment matched at LOCAL_QUORUM. The value verification covers the measured operations and sampled keys.

The treatments differed before scaling: vnode baseline read p95 was 2.240 ms versus 3.033 ms for tablets, approximately **26.1% lower**. Baseline lag p95 also differed (0.006 versus 0.190 s). Identical request sequences therefore did not establish equivalent starting performance. Fixed run order, separate days, cache and maintenance histories, and shared-host activity constrain comparisons from the outset. Windows available memory later fell to 161 MiB during tablet expansion; the available evidence cannot isolate its effect. Phase differences below are descriptive observations of these histories.

Table 3: Client operation latency by measured phase. All values are milliseconds; T denotes tablets and V denotes vnodes. Each row summarizes one run segment, not a replicated treatment estimate.

Treatment / phase	Read mean	Read p95	Read p99	Write mean	Write p95	Write p99
T / Baseline	1.809	3.033	3.989	1.413	2.410	3.098
T / Expansion	1.950	3.493	4.840	1.458	2.604	3.417
T / Four-node	1.896	3.213	4.015	1.474	2.461	3.065
T / Contraction	1.622	2.771	3.764	1.202	2.024	2.729
T / Final three	1.769	3.035	3.830	1.402	2.385	2.918
V / Baseline	1.362	2.240	2.832	1.098	1.800	2.293
V / Expansion	1.655	2.830	3.826	1.228	2.134	2.792
V / Four-node	1.584	2.823	3.712	1.150	1.941	2.448
V / Contraction	1.954	3.425	4.709	1.385	2.470	3.110
V / Final three	1.826	3.153	4.005	1.355	2.323	2.896

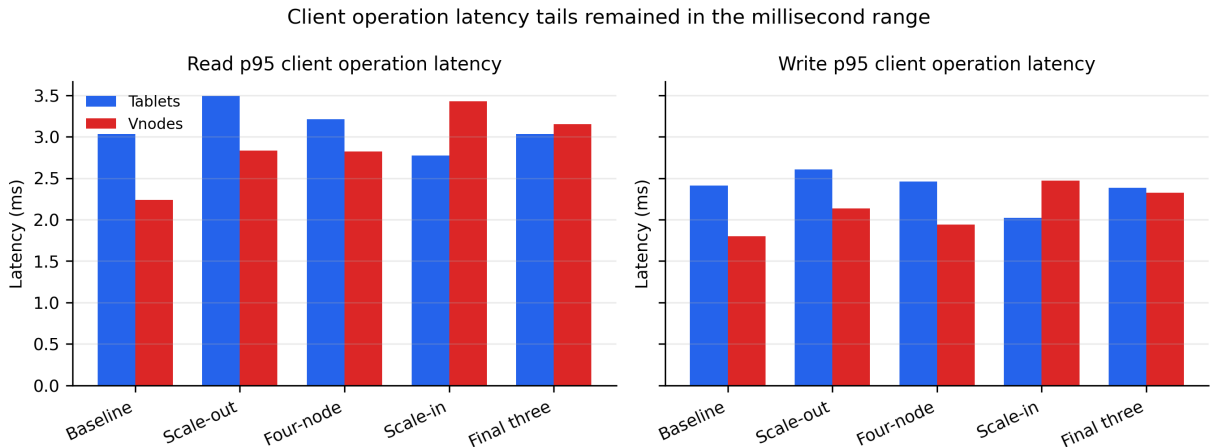


Figure 2: Read and write p95 client operation latency. Millisecond-scale operation times can coexist with seconds of scheduled-start lag because the client issues operations serially.

Vnodes had lower read p95 values in the first three phases, while tablets had the lower value during contraction. Final-three read p95 values were close. These differences characterize the recorded histories. They cannot isolate data distribution from cache, maintenance, treatment order or host activity.

4.2 Placement quiescence and topology timing

Table 4: Command execution and placement-and-counter quiescence are distinct endpoints. Trigger lateness is relative to the first measured request plus 120 seconds.

Transition	Command duration (s)	Quiescence onset (s after command start)	Trigger lateness (s)
Tablet expansion	1.584	51.265	0.214
Vnode expansion	1.486	22.566	0.187
Tablet contraction	15.544	17.691	0.112
Vnode contraction	9.932	11.323	0.159

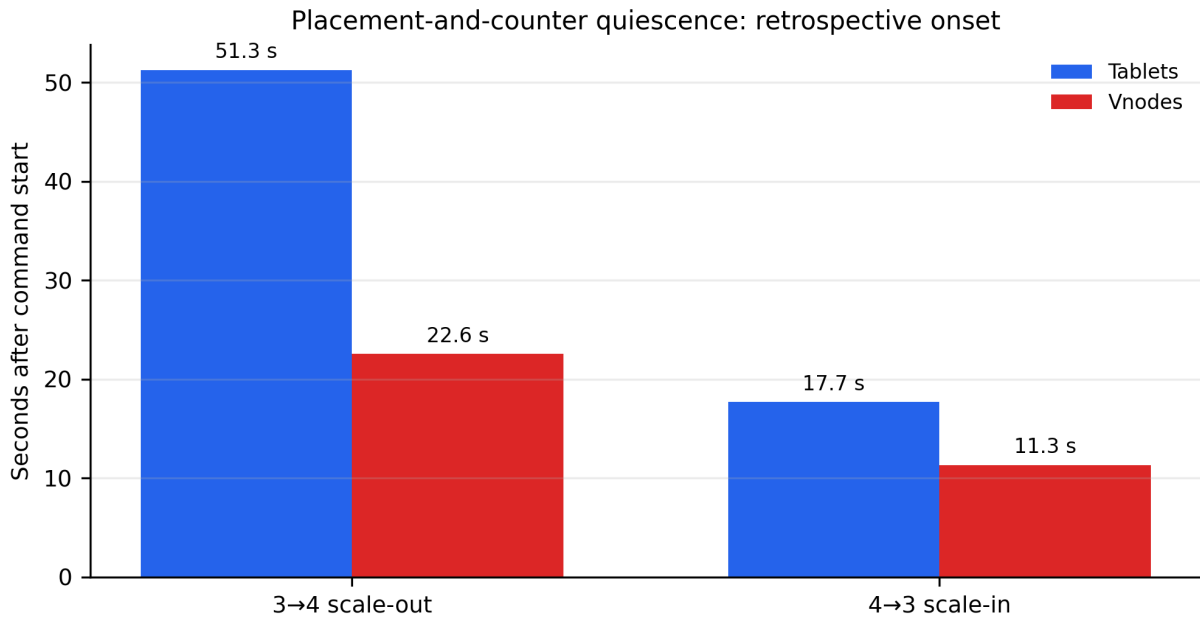


Figure 3: Earliest sampled start of a subsequently verified 60-second unchanged-placement and idle-counter window. The observations are bounded by collection cadence and counter scope.

The criterion was reached earlier by vnodes in these observations, but it measures only placement and the collected counters. In particular, vnode contraction reached it at about 11 seconds, while the largest client backlog developed roughly five minutes later. Tablet file-transfer coverage is also incomplete (Section 4.4). These onsets should therefore be read as partial topology milestones alongside the service and storage results. The ring showed a joining fourth node before all 1,024 token rows became Up/Normal. Vnode logs independently recorded completion of 3,851 bootstrap ranges and streaming completion; the reported bootstrap duration was ten seconds. That internal duration begins later than external command initiation and should not replace the end-to-end observation.

For tablets, expansion changed 32 ranges into 64, producing 128 RF=2 assignments. Contraction returned to 32 ranges and 64 assignments. The first post-merge count of 32 occurred 11.519 seconds after decommission initiation, but a further migration between surviving nodes delayed the placement milestone until 17.691 seconds. For vnodes, the first complete three-node ring sample began at +9.283 seconds, before the last observed counter increase at +10.008 seconds. The joint quiescence window therefore started at the later +11.323-second ring sample. Appendix B gives the observation intervals.

Decommissioning and stopping the departed process were separate events. Collectors continued to observe node 4 after ring departure until it was stopped. Appendix A records the process timing and partial post-decommission statistics.

4.3 Tablet splitting, merging and placement inheritance

Figure 1 summarizes the observed lifecycle. The saved metadata separates a change in range boundaries from a change in replica placement. All 32 original ranges split into two children; each child pair had identical host/shard assignments. Twenty-eight pairs matched the parent’s current assignment in the preceding snapshot, and four matched its pending assignment. Thus all 32 pairs were explained once migration state was included. Treating only the old current assignment as authoritative would have created four false anomalies.

During merging, all 32 final ranges combined two prior ranges. Thirty-one pairs already had identical current host/shard assignments. The apparent exception, ending at token -1, had a pending migration in `use_new` that aligned the second child with the first. The next snapshot recorded the merged parent’s aligned assignment. This resolves the observed placement discrepancy without claiming that sampling captured every internal step.

One sampled intra-node migration moved a replica from shard 0 to shard 1 on node 2, retaining the same two hosts. Its resulting placement matched the neighboring merge partner. This is consistent with merge preparation, although the internal scheduling decision was not logged. After the range count had fallen to 32, a separate migration moved one replica from node 1 to node 2 while retaining node 3. That surviving-node reassignment explains why range-count convergence preceded placement convergence.

The exact-build allocator defaults predict the observed 32/64/32 sequence from eligible shard counts [24]. Small observed replica sizes favor this explanation over a data-size trigger; missing historical runtime settings leave the attribution conditional (Appendix B).

Across the complete lifecycle, the original 32 boundary positions returned, but 11 ranges had different host sets and 12 had different host/shard assignments. The aggregate node counts returned to 21/22/21. Restoration of aggregate balance therefore did not mean restoration of the original placement. By contrast, the initial and final vnode token-owner signatures matched, with 768 tokens and 256 per surviving node.

4.4 Movement observability differs by path

Table 5: Deltas in the collected stream-manager counters. Sender/receiver symmetry is an accounting check for the measured path, with coverage limited to that path.

Directional counter	Tablets (bytes)	Vnodes (bytes)
Expansion: node 4 incoming	369,362	450,160,472
Expansion: original-node outgoing sum	369,362	450,160,472
Contraction: node 4 outgoing	369,362	450,160,472
Contraction: surviving-node incoming sum	369,362	450,160,472

Sender and receiver deltas agreed for each transition, with no decreases in the audited series. Appendix A gives per-node contributions and initial observation delays.

Node 4 received roughly 418 MB of gross Docker traffic early in tablet expansion and held approximately 399 MB of reported replica data. The much smaller stream counter therefore cannot describe the complete physical transfer. Exact-build source inspection shows that the collected counters are advanced through mutation-stream progress paths [25–27], while tablet file streaming has separate stream-blob byte accounting [28, 29]. That implementation difference is consistent with the observed gap. Historical tablet logs and runtime path settings are absent, so the exact excluded byte total and table attribution cannot be reconstructed.

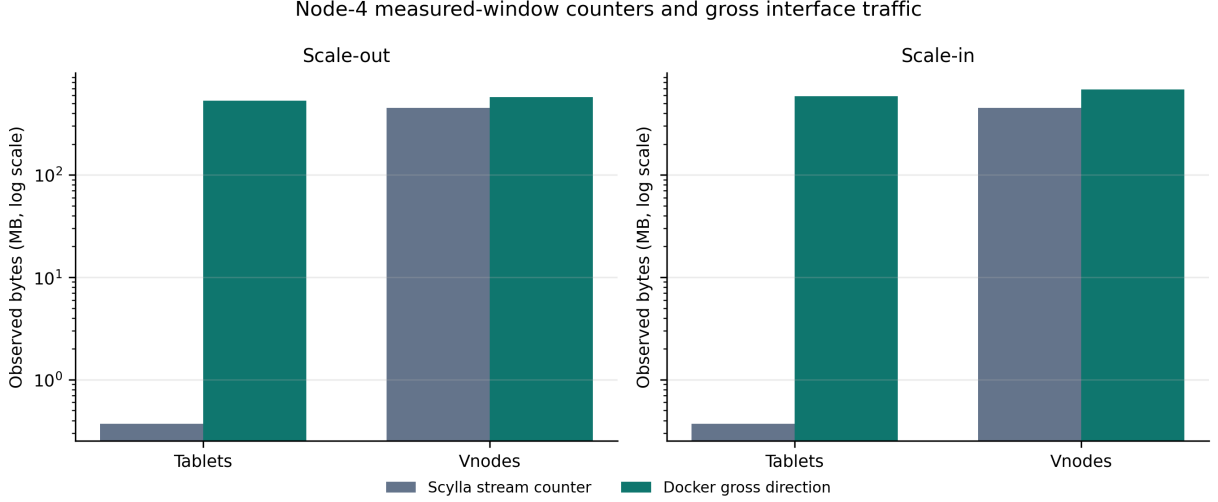


Figure 4: Node-4 directional deltas within measured transition arms: incoming/RX during expansion and outgoing/TX during contraction. Stream counters are 0.369362 MB for tablets and 450.160472 MB for vnodes in each direction. Docker gross deltas are approximately 528.999/577.998 MB during expansion and 590/680 MB during contraction. Sampling endpoints and observation durations differ. These measures have different coverage.

The early vnode receive total of roughly 467 MB is closer to its streaming counter, but also includes unrelated network traffic. Summing interface RX and TX would double-count transfers in some comparisons. A transfer-efficiency ratio would require instrumentation covering the same paths.

4.5 Client scheduling: immediate and delayed effects

Table 6: Scheduled-start lag, in seconds. The two largest maxima arise in opposite transition directions.

Treatment / phase	Lag p95 (s)	Maximum (s)
T / Baseline	0.190	0.334
T / Expansion	21.876	23.070
T / Four-node	0.283	0.496
T / Contraction	0.114	4.425
T / Final three	0.523	0.770
V / Baseline	0.006	0.042
V / Expansion	1.148	8.810
V / Four-node	0.025	0.210
V / Contraction	21.029	23.654
V / Final three	0.455	0.759

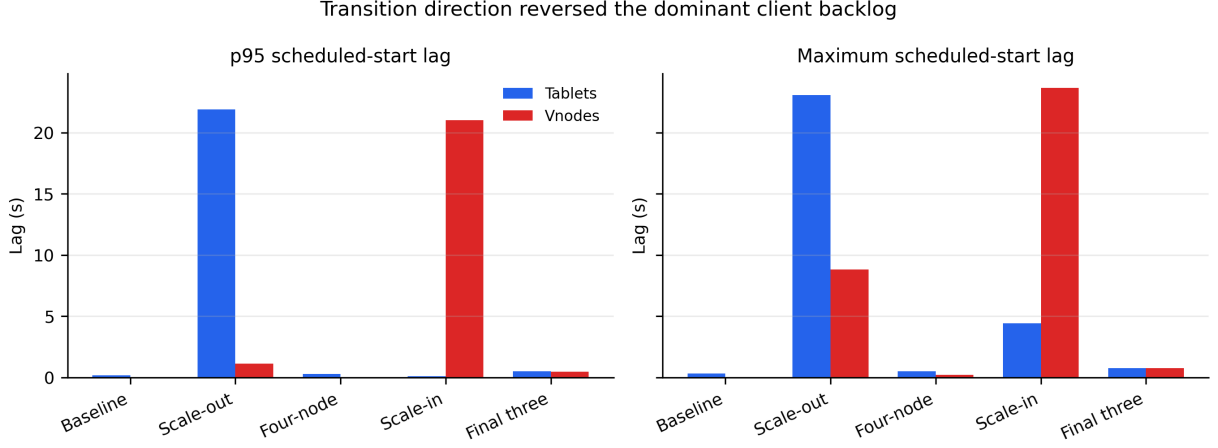


Figure 5: Phase-wide p95 and maximum lag. Mean throughput close to the target obscures temporary backlog followed by catch-up.

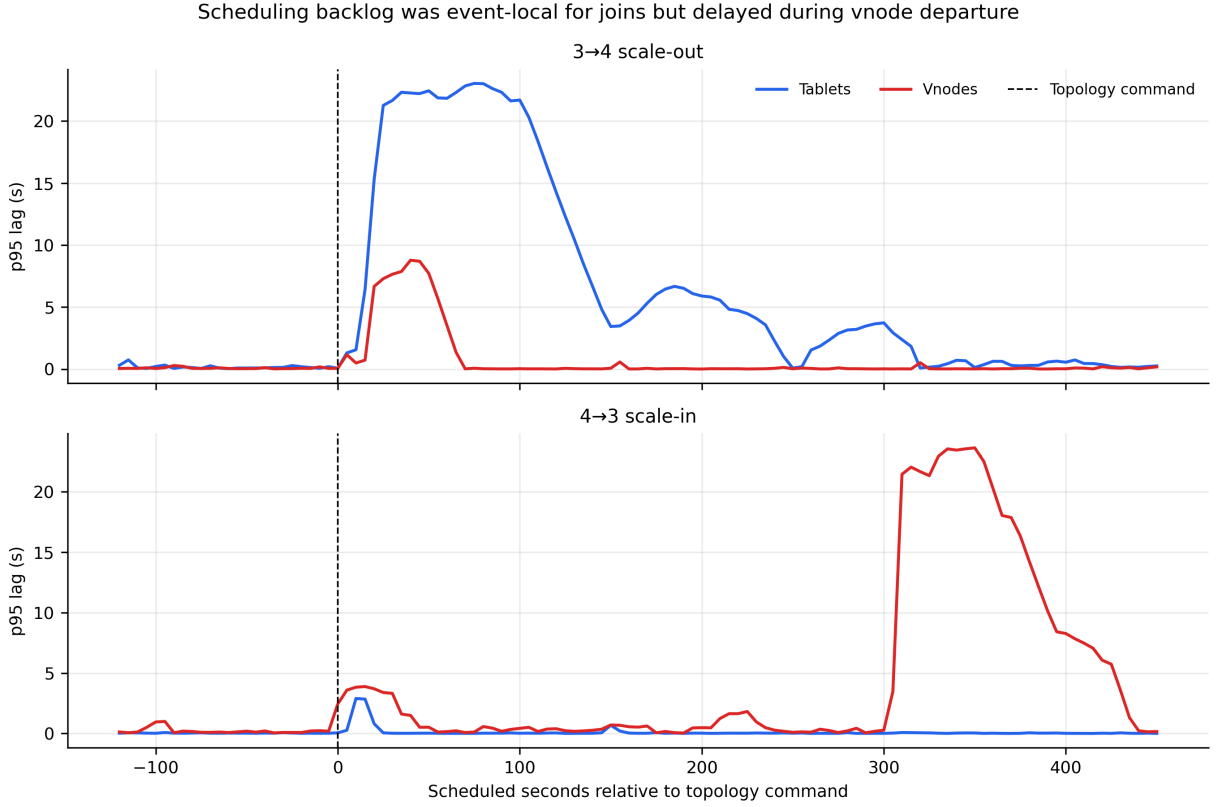


Figure 6: Five-second scheduled-time bins around the topology trigger. The view is cropped to -120 through +450 scheduled seconds; smaller later excursions remain in the complete 900-second records. Delayed requests execute later than their scheduled bin.

For tablet expansion, 15,000 requests scheduled in seconds 90-120 took 29.80 monotonic seconds, including 27.01 seconds inside operations. The next 15,000 took 51.27 seconds, including 49.18 seconds inside operations and 2.08 seconds between them. Thus the service deficit accumulated primarily inside the client operation boundary. Maximum inter-operation gaps were 8.018 and 12.786 ms. The missing tablet server logs prevent assigning this deficit to a specific server-side mechanism. Appendix A gives the independent timing endpoints, including the coincidence between the rounded request-span and topology-onset values. Vnode expansion also accumulated backlog, but less. Its largest five-second scheduled slice took approximately 10.97 seconds, of which 10.69 seconds were timed operations. The maximum phase lag was 8.810 seconds, compared with 23.070 for tablets. This quantifies the transient service deficit in this run.

Vnode contraction had two distinct effects. The immediate post-command interval reached about 3.910 seconds of lag, then recovered. The much larger 23.654-second maximum developed approximately five minutes later. Requests scheduled in seconds 425-435 executed from approximately 15:31:49 to 15:32:20 UTC; successive groups of 2,500 consumed 9.31 and 21.40 seconds inside operations. The slowdown began before node 4 received SIGTERM at 15:32:34.697, ruling out that later stop as its initial cause.

All six surviving-node shards began off-strategy compaction almost simultaneously, approximately 300 seconds after their last logged SSTable receipt. A five-minute trigger in the exact-build source provides an implementation-consistent explanation [25, 30]; Appendix B details the timing and candidate counts.

In a surrounding 35.5-second resource window, nodes 1-3 recorded approximately 2.46 GB of additional block reads and 2.37 GB of writes in total, while node 4's block counters were unchanged. All 16 streaming series were flat. This supports delayed local storage work rather than renewed measured streaming. The short compaction-manager messages ended near 15:31:54, whereas slow operations and later file modification times extended to approximately 15:32:20. The logs therefore do not account for every millisecond of slowdown. Compaction, shared-host I/O and foreground work are associated in time; a controlled causal decomposition is unavailable.

Appendix C quantifies how many post-event requests exceeded lag thresholds. In particular, at least five seconds of lag affected 21.488% of post-event tablet expansion requests and 15.005% of vnode contraction requests. These are request exposure fractions, not outage durations.

4.6 Balance, cleanup and storage history

Table 7: A common logical-placement comparison. RF=2 shares sum to 200%, apart from rounding. Tablet shares use verified equal-width ranges; vnode percentages are the ring's effective ownership reports.

State	Tablet token-space shares, N1/N2/N3/N4 (%)	Vnode reported RF=2 shares, N1/N2/N3/N4 (%)
Initial three	65.625 / 68.750 / 65.625 / -	65.48 / 63.78 / 70.74 / -
Four-node	50 / 50 / 50 / 50	50.86 / 47.08 / 53.40 / 48.67
Final three	65.625 / 68.750 / 65.625 / -	65.48 / 63.78 / 70.74 / -

The four-node tablet endpoint had exactly 32 replicas per node and reported bytes of 400,686,101; 401,861,023; 400,458,341; and 399,372,064. Their coefficient of variation was approximately 0.22%, and maximum-minus-minimum divided by the mean was 0.62%. Logical token-space variation was zero. The vnode four-node ownership coefficient of variation was approximately 4.753%. These characterize balance in this dataset and token realization, within the observed cluster.

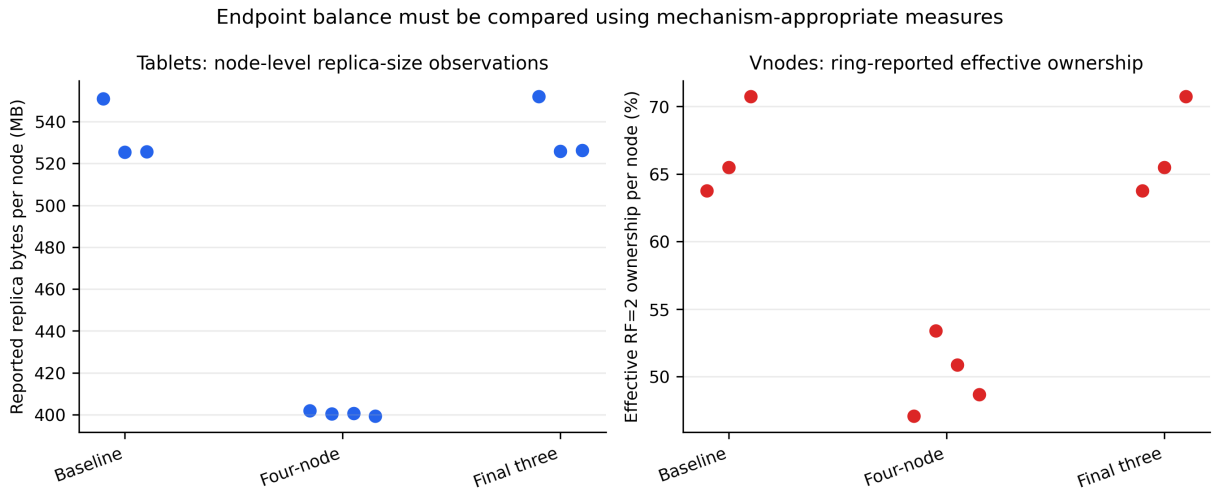


Figure 7: Tablet reported replica-size bytes and vnode effective ownership at lifecycle endpoints. The panels deliberately retain their respective units; Table 7 supplies the direct token-space comparison.

After vnode expansion, the original nodes initially retained their prior live-byte totals. Serial cleanup

then ran for 4.398, 2.939 and 2.882 seconds, a combined 10.219 seconds of command execution. The approximately 180.780-second wall interval also contained 170.561 seconds of checks/operator gaps and is not cleanup service time. Cleanup is relevant after expansion because nodes can retain data for ranges they no longer own [31, 32].

Post-cleanup old-node live bytes were 459,724,966; 417,238,653; and 481,774,985. The reduction of 244,740,998 bytes cannot be assigned wholly to obsolete-data removal because cleanup and flushing changed storage state. Original-node memtables were zero, while node 4 still held 54,148,158 memtable bytes. Accordingly, the subsequent four-node phases were not cache- or memtable-matched.

At the final vnode checkpoint, live bytes were 525,841,391; 523,193,434; and 633,114,947, with 2/4/6 SSTables and partition estimates 491,495/488,925/592,081. The byte spread was 19.60% of the mean. Tablets ended with 21/22/21 replicas and 526,231,881/551,974,438/525,773,936 reported bytes, a spread of 4.90%. The vnode disk totals reflect overlapping file contents and maintenance history as well as ownership. They should not be equated with the logical shares in Table 7.

4.7 Resolving the vnode final partition-count excess

The final vnode estimates summed to 1,572,501, exceeding the nominal 1,500,000 RF=2 node-key pairs by 72,501. To distinguish local file overlap from extra distinct replicas, preserved volumes were mounted read-only and audited with the matching offline SSTable tools [23]. The selected 12 files matched the pre-shutdown 2/4/6-file checkpoint. Two later shutdown-time files per node were excluded; the audit therefore addresses that retained file set, not every file ultimately present in the volumes.

Per-file `rows_count` metadata matched extracted partition-key counts in all 12 files. All IDs were within the dataset range, and no file contained repeated extracted IDs.

Table 8: Exact key accounting for the selected final-checkpoint SSTables. The sum of distinct local IDs counts replicas across nodes, not globally distinct keys.

Node	SSTable entries	Distinct local IDs	Repeated entries across local files
1	491,495	491,495	0
2	488,925	477,873	11,052
3	592,081	530,632	61,449
Total	1,572,501	1,500,000 node-ID pairs	72,501

The union contained exactly 750,000 IDs. Every ID appeared on exactly two nodes; none appeared on one or three. Thus the entire 72,501-entry excess is explained by repeated keys across SSTables on the same node, with no unexplained residual in this selected file set. The distinct local counts also match the initial post-load counts.

The exact accounting identifies local file overlap. Its scope is the selected files; memtables, latest values, timestamps and an atomic historical snapshot were not reconstructed. The separate four-node post-cleanup estimate excess of 135,784 remains unallocated because that historical file set was not retained in the same form. Final-checkpoint overlap does not retrospectively prove its exact composition.

5 Discussion

5.1 The mechanisms expose different notions of completion

The tablet lifecycle involved migration, shard alignment, splitting/merging and subsequent redistribution. Its final boundary count and aggregate replica counts returned to their initial values while individual assignments changed. Vnode token ownership returned to its original signature, but local storage contents still reflected streaming and overlapping SSTables. Logical placement and physical file organization therefore followed different histories even when the cluster returned to three members.

The delayed vnode maintenance result also separates topology convergence from service recovery. An unchanged ring and idle streaming counters establish the specified quiescence condition, but do not imply that all deferred storage work is finished. Conversely, a client can recover from backlog by issuing requests

faster than the target schedule for a period, so a near-target phase average does not imply an undisturbed transition.

5.2 Multiple outcomes and scope

Tablets achieved equal four-node token-space shares and required no manual post-join cleanup in this lifecycle. Vnodes reached the placement-and-counter milestone earlier. Client backlog favored opposite treatments in the two transition directions, while offline extraction resolved the physical-count anomaly. Together these observations show why an elasticity comparison benefits from several endpoints rather than one completion time.

Scope and limitations. One lifecycle per treatment, fixed order, unequal baselines and inter-phase gaps, and vnode-only cleanup limit causal inference. Shared CPU, NVMe storage, filesystem cache and host memory couple all processes; the 161-MiB Windows observation alone cannot establish paging. Effective VM vCPU limits and exact configured memory were unavailable. Vnode logs support a deferred-compaction explanation, whereas the missing tablet logs leave its expansion slowdown without a server-side diagnosis. Source inspection establishes implementation-consistent mechanisms; historical runtime choices remain conditional where settings were not retained. Sampled quiescence covers only the recorded metadata and counters, and independent collector clock offsets were not calibrated. The evidence thus supports a descriptive single-host case study, with treatment-level repeatability and production generalization left open.

5.3 Future work

Repeated, counterbalanced runs on multiple hosts would estimate variability and separate treatment from execution history. Effective runtime configuration and symmetric server logs would strengthen mechanism attribution. Controlled maintenance and dedicated transfer-path instrumentation could isolate deferred storage effects and compare migration bytes on a common basis. Workload concurrency, replication and dataset size are further dimensions for testing the findings.

6 Conclusion

Two workload-matched ScyllaDB lifecycles completed 1.08 million requests each without reported workload failures and retained 750,000 logical keys. Tablets adapted range boundaries and replica placement; vnodes restored token ownership while retaining a different local file history. The main distinction is the separation between placement quiescence, deferred maintenance and client service: an early topology milestone can precede a substantial later backlog.

The audit explains the observations at several levels. Pending assignments reconcile tablet split/merge inheritance, implementation-specific paths explain why stream counters cannot be compared as total movement, and vnode logs associate deferred work with a later slowdown. Exact offline key extraction accounts for all 72,501 excess final SSTable entries as overlap across files on the same node. These findings establish a concrete baseline for deeper, replicated studies of scaling behavior.

7 Data and code availability

The workload, monitoring and analysis scripts, Docker definitions, derived summaries and figure sources are available at the project repository [33]. The approximately 949 MiB of raw observations is not included in the repository; it is available from the author on request, with hashes in the manifest for verification. Appendix D describes the preserved evidence.

A Measurement and evidence audit

A.1 Clocks and client accounting

Across all ten measured request files, indices were valid, monotonic starts increased strictly, and no inter-operation gap was below -0.01 ms. UTC starts nevertheless stepped backward 38 times in tablet files and 50 in vnode files; 83 UTC operation spans were negative. The largest UTC backstep was 2.157 ms, and the largest adjacent change in UTC-versus-monotonic drift was 5.354 ms. UTC spans were 9.450-16.834 ms shorter than monotonic spans in 120-second phases and 71.824-92.904 ms shorter in 900-second phases. These small wall-clock effects cannot explain the multi-second monotonic scheduling backlogs.

The workload’s completion banner covers final CSV/state persistence but not every setup or teardown step. For example, the vnode final warmup banner reported 485.92/s, whereas its first-request-to-last-completion span was 30.204 seconds, approximately 496.61/s. These values use different timing boundaries. UTC-derived request-span rates are retained as descriptive summaries; they are not substituted for monotonic lag or operation duration.

The apparently identical 51.265-second values have different endpoints and clocks. The tablet topology interval runs from the join command at 17:41:59.0830105 UTC to the first accepted observation at 17:42:50.347804 UTC: 51.2647935 seconds, rounded to 51.265. The recorded raw-request audit gives 51.265243 monotonic seconds for the 15,000 requests scheduled in seconds 120-150. Their independent UTC endpoints, 17:41:58.859705 and 17:42:50.120515, span 51.260810 seconds and are corroborated by the archived five-second bins. The near equality is coincidental. Monotonic accounting decomposes that request window into 49.182314 seconds inside operations and 2.082929 between operations; the preceding window decomposes into 27.012917 and 2.785911 seconds.

A.2 Coverage and parser corrections

Table A.1: Saved observations within measured request intervals. Each collector also bracketed the interval. Sampling counts need not equal duration because commands take time and cadence varies.

Phase	Tablet snapshots / stream / Docker samples	Vnode ring / stream / Docker samples
Baseline	115 / 120 / 35	59 / 120 / 36
Expansion	857 / 897 / 261	436 / 896 / 266
Four-node	114 / 120 / 35	58 / 120 / 36
Contraction	863 / 900 / 270	435 / 893 / 259
Final three	114 / 120 / 35	58 / 120 / 35

Nine tablet size snapshots were incomplete: seven during expansion and two during contraction. They contained 12 missing-replica reports, ten at `end_migration` and two at `cleanup`. Every missing host was the outgoing replica. Token sets still matched. The destination had no reported size in the same snapshot in all 12 cases, but appeared with a positive size, current placement and a cleared transition in the immediately following snapshot, 1.021-1.087 seconds later. Three cases crossed the split, so their next sizes refer to child ranges rather than an unchanged parent. Incomplete samples were excluded from byte-balance endpoints and never interpreted as zero-size or unavailable replicas.

The ring parser accepts unknown load (?) as well as numeric sizes: seven node-4 snapshots contain this form, with all 256 node-4 rows present. All 5,358 raw ring-file hashes and address counts passed validation. Docker size parsing accepts scientific notation such as `1e+03MB`. The SSTable key extractor handles records crossing input-chunk boundaries and was validated against all 12 selected files’ metadata, including a 5,555-key test case.

The vnode expansion contributions were 135,370,928, 153,890,512 and 160,899,032 bytes from nodes 1-3. Contraction recorded the same amounts in the reverse direction. Tablet expansion contributions were 46,152, 184,898 and 138,312 bytes. The audited series showed no counter decreases. Node 4 was not initially observable, however: its first metrics arrived approximately 7.587 seconds after tablet join initiation and 6.805 seconds after vnode initiation.

A.3 Acquisition and operational qualifications

The complete tablet record contains 6,947 snapshot-index entries, 7,160 complete streaming samples and 7,194 Docker rows. The vnode record contains 5,358 indexed ring samples, 10,976 complete streaming samples and 10,794 Docker rows. Event logs contain 16 tablet and 22 vnode records, including five phase starts and ends per treatment and vnode cleanup events.

Health checks reported zero pending tasks and no outgoing streams at the stated checkpoints. Separate container inspections reported no restarts or OOM flags at inspection times; the Docker CSV did not continuously record those fields. Startup, cleanup and departure warnings are preserved in vnode logs. Reclaim warnings of 44, 76 and 124 ms occurred during node-1 cleanup outside the measured expansion arm. Transient gossip/raft messages accompanied membership changes, and SIGTERM/abort messages accompanied stopping node 4. Their timing is documented without declaring every warning harmless.

Decommissioning and stopping the departed process were separate events. The fourth process remained observable after leaving the ring, so resource and streaming collectors legitimately continued to report four processes. Confirmed process absence occurred about 149.962 seconds after tablet decommission initiation and 356.808 seconds after vnode initiation. Unchanged three-node placement was verified in observations taken after those stops. A failed post-decommission `tablestats` call is not a zero-partition reading; tablets returned an HTTP 500 closed snapshot-gate error, while vnodes returned exit code 4 with partial output.

Inter-phase gaps between one phase-end marker and the next phase-start marker were approximately 1,137/761/1,456/553 seconds for tablets and 3,574/1,541/1,552/1,290 seconds for vnodes. They include checks and warmups and are not pure idle durations. The repeated vnode expansion warmup improved from 470.16/s with 1.771-second maximum lag to 498.52/s with 0.258-second maximum lag. Neither warmup was included as a measured replicate.

Summed observed container-memory peaks were approximately 6.12/6.20 GiB for tablet/vnode expansion and 6.32/6.42 GiB for contraction. Maximum sampled single-container CPU percentages were 189.60/86.76 during expansion and 142.75/144.50 during contraction; Docker percentages can exceed 100% for a multi-core process. These resource samples localize activity but do not establish its cause.

B Topology reconstruction and temporal precision

The tablet expansion predecessor was recorded at +50.232 seconds with 32 ranges and 61 of 64 size observations; the selected quiescence-onset snapshot at +51.265 had 64 ranges and 128 observations. During contraction, a cleanup state at +16.651 preceded a cleared assignment at +17.691. Tablet timestamps were recorded after sequential collection rather than marking a common atomic acquisition instant, so their roughly one-second gaps are observational brackets, not strict physical-time bounds.

For vnode expansion, sample 2195 ran from +20.506 to +21.111 seconds and was not all normal. Sample 2196 ran from +22.566 to +23.227 and was all normal. During contraction, the first complete three-node sample ran from +9.283 to +9.804, but the selected joint-quiescence sample ran from +11.323 to +12.026. The independently observed last counter increase at +10.008 rules out using the earlier ring observation as the joint endpoint.

A candidate-window scan rejected earlier observations because of state, placement or changing counters. In expansion/contraction order, tablet rejection counts were 48/13 for state and 0/4 for placement; vnode counts were 10/4 for state and 0/1 for changing counters. The accepted placement windows spanned at least 60 seconds, as did the aligned counter windows. This retrospective check avoids selecting a visually convenient first normal snapshot.

Table B.1: Changes between sampled metadata states. Splits and merges alter range identity and inflate assignment additions/removals; these are not physical transfer counts.

Metadata measure	Tablet expansion	Tablet contraction
Steps with changed metadata	25	14
Steps with changed placement	8	11
Tablet-count changes	1	1
Shared-boundary assignment changes, summed	16	34
Boundaries added / removed	32 / 0	0 / 32
Assignment additions / removals, summed	80 / 16	34 / 98

Metadata measure	Tablet expansion	Tablet contraction
Endpoint assignment additions / removals	80 / 16	16 / 80

The exact-build allocator source provides a further explanation of the count sequence [24]. Its default minimum of ten replicas per eligible shard, divided by RF=2 and rounded to a power of two, predicts 32 tablets for six eligible shards and 64 for eight; excluding the draining node returns the calculation to 32. The observed replica-size means near the decisions were only 23.872 MiB and 11.939 MiB, far below the source’s 5-GiB target and size-based split threshold. No explicit initial-count override was found in retained configuration/schema evidence. This makes the per-shard minimum a source-supported explanation, subject to missing historical allocator inputs: runtime configuration and tablet decision logs were not preserved.

All six surviving-node shards began off-strategy compaction within a 48-ms interval, approximately 299.805-299.860 seconds after their last logged receipt of table SSTables. Each shard reported 11 received SSTables and 11 compaction candidates. Exact-build source contains a five-minute off-strategy trigger and rearming behavior [25, 30], providing a mechanism consistent with this synchronized delay. Matching counts do not prove the identity of every candidate file or the exact timer anchor.

The intra-node merge-alignment example ended at token 6341068275337658367. Node 2 changed shard 0 to 1 while node 3 remained on shard 1, matching the adjacent range ending at 6052837899185946623. The post-merge surviving-node migration ended at 576460752303423487. Four split parents inherited pending placement at boundaries -1729382256910270465, -1152921504606846977, -576460752303423489 and 4035225266123964415. Their sampled stages were respectively `write_both_read_new`, `cleanup`, `end_migration` and `cleanup`.

The merge exception combined boundaries -288230376151711745 and -1. The first child had already moved from node 4 to node 1. The second moved from node 4 to node 3 and then had a pending node-1 assignment in `use_new`. The merged range retained nodes 2 and 1. The source’s minimum-replica calculation explains why splitting and merging were plausible at small table sizes, but source defaults and recorded states remain different kinds of evidence.

C Backlog exposure

Table C.1: Fractions of requests starting after the recorded command start whose monotonic scheduled-start lag met each threshold. Slightly different denominators reflect actual trigger timing.

Transition	Post-event requests	Lag at least 1 s	At least 5 s	At least 10 s	At least 20 s
Tablet expansion	389,897	42.397%	21.488%	13.895%	10.078%
Tablet contraction	389,943	3.912%	0%	0%	0%
Vnode expansion	389,901	5.901%	4.476%	0%	0%
Vnode contraction	389,920	25.548%	15.005%	10.205%	6.011%

The longest consecutive runs at or above five seconds were 62,967 requests spanning 125.922 seconds of actual starts for tablet expansion, 17,453 spanning 34.882 seconds for vnode expansion, and 58,507 spanning 116.993 seconds for vnode contraction. The first and last affected completion intervals were approximately +23.668 to +218.595, +26.833 to +61.715, and +314.867 to +431.860 seconds respectively, using first start and last completion. No tablet contraction request crossed that threshold.

At the one-second threshold, the last affected completion occurred at +772.552 seconds for tablet expansion, +776.085 for tablet contraction, +66.816 for vnode expansion and +550.062 for vnode contraction. Those endpoints include later excursions and must not be described as uninterrupted event-caused outages. The thresholds were retrospective diagnostics, not preregistered service-level objectives.

D Reproducibility and preserved evidence

The database build ID was `b14b47fb9cb96594eea92615544ef9e54695658a`; source inspection used commit `10c95df28f74`. The workload SHA-256 was `10e8665f776bdb0ff5b182087181bb76f1bd2b4c9cfbcde3a7326065f3c69090`, matching both treatments’ recorded manifests and the inspected implementation. Phase seeds, dataset seed, rate, durations and read percentage were saved in the run plans. Both setup monitor images contained Python 3.12.14 and scylla-driver 3.29.11; their digests differ, so this verifies the dependencies but not the identity of every historical workload container.

The repository [33] contains the workload, loading and monitoring scripts (`monitoring/`), the Docker Compose definitions (`docker/`), the primary analysis and focused-check scripts, and the derived outputs (`data/analysis/primary-comparison/`): 11 manifest-listed files plus `manifest.json`, all 23 structural checks passing and output hashes matching the manifest. Figure data and the script that regenerates the figures are in `figures/`. The archived generator reports version 1.0.0 and the repository contains version [state which]; clean regeneration with the exact historical generator has not been demonstrated. The approximately 949 MiB of raw observations is not included because of hosting limits; it is preserved by the author, and the manifest hashes allow verification of any copy supplied. The offline SSTable audit is represented in the repository by [the extraction script and per-file outputs / only the statistics file for the first node of the first audited set]; Table D.1 therefore [can be regenerated from the repository / is reported from the preserved volumes].

All four volumes per treatment were preserved after container removal [34, 35], and eight vnode stdout/stderr log files were retained. The offline final-file audit selected the pre-shutdown set using the recorded inventory and modification times, with a cutoff before later shutdown-created files. Table D.1 lists the per-file counts; filenames omit the common `mt-` prefix and `-big-Data.db` suffix.

Table D.1: Exact selected-file key counts supporting Table 8. Later shutdown-time files and memtables are outside this audit.

Node	SSTable identifier	Extracted keys and metadata rows
1	3h41_175f_12sls237btplviwbv3	244,768
1	3h41_175f_19nio1zlu4wzz7hwjz	246,727
2	3h41_16wl_1bd8w2kmpdamz3rbg8	5,555
2	3h41_16wl_1bsog2ho5lbb8yasoo	5,497
2	3h41_175f_1axtc2ho5lbb8yasoo	239,115
2	3h41_175f_1inlc2kmpdamz3rbg8	238,758
3	3h41_16wl_213682ih550kb33z63	30,836
3	3h41_16wl_2qdo01ya2y59xxtxhn	30,613
3	3h41_16wm_1p32o1ya2y59xxtxhn	52,163
3	3h41_16wn_3elkw1ya2y59xxtxhn	147,218
3	3h41_1755_042ls1ya2y59xxtxhn	65,383
3	3h41_175f_1inlc2ih550kb33z63	265,868

References

- [1] ScyllaDB. Data Distribution with Tablets. <https://docs.scylladb.com/manual/stable/architecture/tablets.html>, 2026. Accessed September 2026.
- [2] ScyllaDB. Nodetool ring. <https://docs.scylladb.com/manual/stable/operating-scylla/nodetool-commands/ring.html>, 2026. Accessed September 2026.
- [3] Giuseppe DeCandia et al. Dynamo: Amazon’s Highly Available Key-value Store. In *Proceedings of SOSP*, 2007. URL <https://www.amazon.science/publications/dynamo-amazons-highly-available-key-value-store>.
- [4] Apache Cassandra. Dynamo: Architecture and Virtual Nodes. <https://cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html>, 2026. Accessed September 2026.
- [5] Fay Chang et al. Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of OSDI*, 2006. URL <https://www.usenix.org/legacy/event/osdi06/tech/chang/chang.pdf>.

- [6] Avi Kivity. How We Implemented ScyllaDB’s “Tablets. <https://www.scylladb.com/2024/06/17/how-tablets/>, 2024. Accessed September 2026.
- [7] Konstantin Osipov. ScyllaDB’s Safe Topology and Schema Changes on Raft. <https://www.scylladb.com/2024/06/18/scylladbs-safe-topology-and-schema-changes-on-raft/>, 2024. Accessed September 2026.
- [8] Jörn Kuhlenkamp, Markus Klems, and Oliver Röss. Benchmarking Scalability and Elasticity of Distributed Database Systems. *Proceedings of the VLDB Endowment*, 7(13):1219–1230, 2014. URL <https://www.vldb.org/pvldb/vol17/p1219-klems.pdf>.
- [9] Saneyasu Yamaguchi and Yuki Morimitsu. Improving Dynamic Scaling Performance of Cassandra. *IEICE Transactions on Information and Systems*, E100.D(4):682–692, 2017. URL <https://doi.org/10.1587/transinf.2016DAP0009>.
- [10] Ying Liu, Daharewa Gureya, Ahmad Al-Shishtawy, and Vladimir Vlassov. OnlineElastMan: self-trained proactive elasticity manager for cloud-based storage services. *Cluster Computing*, 20:1977–1994, 2017. URL <https://doi.org/10.1007/s10586-017-0899-z>.
- [11] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of SoCC*, 2010. URL <https://www.cs.albany.edu/~jhh/courses/readings/cooper.socc10.benchmarking.pdf>.
- [12] ScyllaDB. `tools/scylla-nodetool.cc` (estimated_partition_count) and `api/column_family.cc` (get_estimated_row_count), commit 10c95df28f74. <https://github.com/scylladb/scylladb/blob/10c95df28f74/tools/scylla-nodetool.cc>; https://github.com/scylladb/scylladb/blob/10c95df28f74/api/column_family.cc, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [13] ScyllaDB. Nodetool flush. <https://docs.scylladb.com/manual/stable/operating-scylla/nodetool-commands/flush.html>, 2026. Accessed September 2026.
- [14] Intel. Intel Core i5-13450HX Processor Specifications. <https://www.intel.com/content/www/us/en/products/sku/232132/intel-core-i513450hx-processor-20m-cache-up-to-4-60-ghz/specifications.html>, 2026. Accessed September 2026.
- [15] ScyllaDB. Release build index: 2026.3.0, x86-64, build b14b47fb9cb96594eea92615544ef9e54695658a. <https://backtrace.scylladb.com/releases.html>, 2026. Accessed September 2026.
- [16] ScyllaDB. Consistency Levels. <https://docs.scylladb.com/manual/stable/cql/consistency.html>, 2026. Accessed September 2026.
- [17] ScyllaDB. Data Definition. <https://docs.scylladb.com/manual/stable/cql/ddl.html>, 2026. Accessed September 2026.
- [18] ScyllaDB. Glossary: RF-rack-valid keyspace. <https://docs.scylladb.com/manual/stable/reference/glossary.html#term-RF-rack-valid-keyspace>, 2026. Accessed September 2026.
- [19] ScyllaDB. System Requirements. <https://docs.scylladb.com/manual/branch-2026.3/getting-started/system-requirements.html>, 2026. Accessed September 2026.
- [20] Docker Inc. docker container stats. <https://docs.docker.com/reference/cli/docker/container/stats/>, 2026. Accessed September 2026.
- [21] ScyllaDB. Metrics: `scylla_streaming_total_incoming_bytes` and `scylla_streaming_total_outgoing_bytes`. <https://docs.scylladb.com/manual/stable/reference/metrics.html>, 2026. Accessed September 2026.
- [22] ScyllaDB. Nodetool tablestats. <https://docs.scylladb.com/manual/stable/operating-scylla/nodetool-commands/tablestats.html>, 2026. Accessed September 2026.
- [23] ScyllaDB. ScyllaDB SSTable, branch 2026.3, offline dump-statistics and dump-data operations. <https://docs.scylladb.com/manual/branch-2026.3/operating-scylla/admin-tools/scylla-sstable.html>, 2026. Accessed September 2026.

- [24] ScyllaDB. `service/tablet_allocator.cc` (eligible shards, per-shard minimum, sizing candidates and power-of-two decisions), `db/config.cc` (defaults), `db/tablet_options.hh` (default power-of-two setting), and `service/tablet_allocator_fwd.hh` (5-GiB target), commit 10c95df28f74. ; ; . https://github.com/scylladb/scylladb/blob/10c95df28f74/service/tablet_allocator.cc; <https://github.com/scylladb/scylladb/blob/10c95df28f74/db/config.cc>; https://github.com/scylladb/scylladb/blob/10c95df28f74/db/tablet_options.hh; https://github.com/scylladb/scylladb/blob/10c95df28f74/service/tablet_allocator_fwd.hh, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [25] ScyllaDB. `streaming/stream_session.cc`, commit 10c95df28f74, `offstrategy_trigger` and `mutation_fragment_receiver`. https://github.com/scylladb/scylladb/blob/10c95df28f74/streaming/stream_session.cc, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [26] ScyllaDB. `streaming/stream_manager.cc`, commit 10c95df28f74, `metric` registration and `update_progress`. https://github.com/scylladb/scylladb/blob/10c95df28f74/streaming/stream_manager.cc, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [27] ScyllaDB. `streaming/stream_transfer_task.cc`, commit 10c95df28f74, `mutation-stream` progress callback. https://github.com/scylladb/scylladb/blob/10c95df28f74/streaming/stream_transfer_task.cc, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [28] ScyllaDB. `streaming/stream_blob.cc`, commit 10c95df28f74, `tablet_stream_files`, `stream-blob` receiver and returned file-byte accounting. https://github.com/scylladb/scylladb/blob/10c95df28f74/streaming/stream_blob.cc, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [29] ScyllaDB. `service/storage_service.cc` and `db/config.cc`, commit 10c95df28f74, `tablet` file-stream selection and configuration default. ; . https://github.com/scylladb/scylladb/blob/10c95df28f74/service/storage_service.cc; <https://github.com/scylladb/scylladb/blob/10c95df28f74/db/config.cc>, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [30] ScyllaDB. `replica/table.cc`, commit 10c95df28f74, `timer` methods and `constructor` callback. <https://github.com/scylladb/scylladb/blob/10c95df28f74/replica/table.cc>, 2026. Source inspection at commit 10c95df28f74; accessed September 2026.
- [31] ScyllaDB. Adding a New Node Into an Existing ScyllaDB Cluster (Out Scale). <https://docs.scylladb.com/manual/stable/operating-scylla/procedures/cluster-management/add-node-to-cluster.html>, 2026. Accessed September 2026.
- [32] ScyllaDB. Nodetool cluster cleanup. <https://docs.scylladb.com/manual/stable/operating-scylla/nodetool-commands/cluster/cleanup.html>, 2026. Accessed September 2026.
- [33] Giorgi Gogsadze. Scylla Research. GitHub repository, 2026. URL <https://github.com/GiorgiGogsadze/Scylla-Research>.
- [34] Docker Inc. Define and manage volumes in Docker Compose. <https://docs.docker.com/reference/compose-file/volumes/>, 2026. Accessed September 2026.
- [35] Docker Inc. docker compose down. <https://docs.docker.com/reference/cli/docker/compose/down/>, 2026. Accessed September 2026.